

Добро пожаловать в Typst!

Typst – это современный язык разметки для сборки документов. Он использует синтаксис, похожий на древнее зло и манящего демона, его величества Markdown, однако с мощной системой функций.

Обычный текст пишется как есть, а для форматирования используются:

- Markdown-подобные маркеры для текста
- Функции для всего остального

Форматирование текста

Изменение текста

Результат	Исходный код
Заголовок 1. уровня	= Заголовок
Заголовок 2. уровня	== (под)Заголовок
Жирный	*текст*
<i>Курсивный</i>	_текст_
Моноширинный (inline)	`код`
<u>Подчёркнутый</u>	#underline[текст]
Зачёркнутый	#strike[текст]

Комментарии в Typst напоминают комментарии как в C, C++ и Rust:

Результат	Синтаксис
Однострочный комментарий	// Комментарий в одну строку
Многострочный комментарий	/* Комментарий в несколько строк */

Списки в формате Markdown

Результат	Синтаксис
• Элемент 1 • Элемент 2 ▸ Вложенный элемент1 – 2вложенный элемент1 ▸ Вложенный элемент2 ▸ Вложенный элемент3	- Элемент 1 - Элемент 2 - Вложенный элемент1 - 2вложенный элемент1 - Вложенный элемент2 - Вложенный элемент3
1. Нумерованный список 2. Второй пункт	1. Нумерованный список 2. Второй пункт

Спецсимволы

Вы заметили, что некоторые синтаксические элементы используют спецсимволы, например, # ? Если написать просто # в тексте документа, получите ошибку компиляции. Чтобы ввести спецсимвол, его нужно *экранировать* при помощи символа \: \#.

Символ \ также является спецсимволом.

Полный список спецсимволов приводится не будет, так как их очень много.

Единицы измерения

В Typst все размеры работают через тип `length`, а также через специальный тип `fraction` для единицы `fr`. Единицы пишутся слитно:

2cm, 10pt, 0.5fr

Единица	Тип	Физическое значение	Где применять
pt	Абсолютная	$\frac{1}{72}$ дюйма	Базовая единица Typst. Размеры шрифтов, межстрочные интервалы
cm, mm	Абсолютная	Метрические	Поля страницы, ширина/высота изображений, отступы
in	Абсолютная	Дюйм 25.4 mm	Редко, в основном для совместимости с imperial-системой
px	Абсолютная	$\frac{1}{96}$ дюйма (CSS-пиксел)	Веб-ориентированные макеты. 1px $\approx$ 0.75pt
em	Относительная	Текущий размер шрифта	Масштабируемые inline-отступы, иконки, кнопки
ex	Относительная	Высота строчной x в текущем шрифте	Точная вертикальная подгонка по высоте букв
lh	Относительная	Текущая высота строки (line-height)	Вертикальные ритмы, отступы между абзацами
%	Относительная	Доля родительского контейнера в процентах	width: 50%, height: 100%
fr	Гибкая (fraction)	Доля свободного пространства	Сетки, колонки, растягивание элементов (аналог CSS fr)

`fr` не физическая длина, а инструкция для компоновщика: “раздели оставшееся пространство пропорционально”:

```
#grid(  
  columns: (3cm, 2fr, 1fr),  
  stroke: black,  
  inset : 2mm  
)[Фиксировано][66% остатка][33% остатка]
```

Фиксировано	66% остатка	33% остатка
-------------	-------------	-------------

Спецсимволы # и []

Спецсимвол

Спецсимволом `#` – это главный переключатель между “обычным текстом” и “кодом Typst”. Пока компилятор (или лексический анализатор?) не найдёт `#`, исходный текст до `#` считается простым текстом. Но исходный текст после `#` считается функцией, переменной, символом, условием или вычислением. Компилятор выполнит это и вставит результат выполнения в документ.

Что делает	Пример кода	Результат
Вызов функции	<code>#set page(margin: 2cm)</code>	Меняет отступы страницы
Вставка символа	<code>#sym.alpha, #sym.Lambda</code>	α , Λ
Вставка переменной	<code>#let имя = "Алекс"</code> <code>Привет, #имя!</code>	Привет, Алекс!
Вычисление	Результат: <code> #(12 * 3.5)</code>	Результат: 42
Условная логика	<code>#if (2 * 2 == 4) [Видно]</code> <code>else[Скрыто]</code>	Видно
Блоки содержимого	<code>#emph[Курсивный текст]</code>	<i>Курсивный текст</i>

Проще говоря, **# в тексте** обозначает какой-то технический элемент. Технический элемент внутри другого технического элемента **#** не обозначается: Код: `#figure(table(columns : 2, [col1], [col2]), caption: [Пример \#])`

Результат:

col1	col2
------	------

Table 1: Пример использования#

table тут через # не обозначается, потому что table вызывается в другой функции

Квадратные скобки []

Квадратные скобки обозначают контент.

Код: `#box(fill: color.rgb("#c55050"), inset: 4pt,)[Контент]` – в квадратных скобках содержится контент box. В круглых – параметры box.

Результат :

Контент

Функции внутри [] вызываются через #:

Код:

```
#box(fill: color.rgb("#c55050"), inset: 4pt)[#table(
  columns: 2,
  [Контент 1го столбца], [Контент 2го столбца],
  [Контент 3го столбца]

  #table(columns : 3, [col1], [*col2*], [_*col3*_])
  ],
  [Контент 4го столбца]

  #box(stroke: stroke(paint: color.aqua, thickness: 1pt), inset: 4pt)[Контент внутри
4го столбца!]
  ],
)]
```

Результат:

Контент 1го столбца			Контент 2го столбца
Контент 3го столбца			Контент 4го столбца
col1	col2	col3	Контент внутри 4го столбца!

Итог

Получается, что # обозначает технический элемент, а [] - контент. Их можно и даже нужно комбинировать.

Вызов функции

В общем случае функция вызывается следующим образом:

ИмяФункции(параметры) [контент]

Повторюсь, если Вы хотите вызвать функцию внутри [], нужно поставить перед её именем #.

Элемент	Назначение	Пример
ИмяФункции	Идентифицирует функцию	text
()	параметры	(fill: red, size : 14pt)
[]	контент, тело	[Hello, World!]

Итого получится: `text(fill: red, size : 14pt)[Hello, World!]`

Hello, World!

Вызывать в блоке с контентом надо через #:

```
#text(fill: red, size : 14pt)[Hello, World!]
```

Далее этот аспект будет полагаться понятным и будет опускаться.

Вообще, [] непосредственно после круглых скобок при вызове функции передаётся в функцию как аргумент под названием content.

То есть формат вызова

```
#quote[Цитата оформляется без лишних скобок] “Цитата оформляется без лишних скобок”
```

эквивалентен

```
#quote([Цитата оформляется с лишними скобками]) “Цитата оформляется с лишними скобками”
```

Тут только аргументы разные в угоду семантики.

Вызов функции обязательно обладает именем функции и круглыми скобками, возможно пустыми: `#pagebreak()` — имя и () обязательны, без них функция не будет вызвана.

Константы и свои функции

Константы

Код `#let имя = значение` создаёт константу. Это полезно.

Вообще, эта штука формально называется *иммутабельной привязкой*.

Эта константа доступна всюду ниже её объявления.

Теперь значение этой константы нельзя изменить, исключая вложенные блоки `{}`. Это всё же не совсем константа в её классическом понимании.

```
#let имя = "Адам"
#set имя = "Ева" // <- ошибка
```

Однако, можно затенить константу и такой код будет работать:

```
#let имя = "Адам"
#let имя = "Ева" // <- Ok
```

```
#имя // Ева
```

Функции

Можно написать и свои функции

```
#let great_author(author, year : 2024) = [Привет, #author из #year!]
```

Привет, Павел из 2025!

```
#let warning-box(content) = {
  let icon = "⚠"
  let bg = yellow.lighten(80%)
  box(fill: bg, inset: 1em, radius: 4pt)[
    #icon #strong[Внимание:] #content
  ]
}
#warning-box()[Не забудьте проверить расчёты!]
```

 **Внимание:** Не забудьте проверить расчёты!

Функция так же может возвращать и значение определённого типа:

```
#let offset(val: length) = (val - 2)

#table(
  columns: 5,
  rows: 2,
  [$x$], [1], [2], [3], [4],
  [offset(val : $x$)], [offset(val: 1)], [offset(val: 2)], [offset(val: 3)],
  [offset(val: 4)],
)
```

x	1	2	3	4
-----	---	---	---	---

offset(val : x)	-1	0	1	2
--------------------	----	---	---	---